

📄 Análise de Código — PesquisAI v0.2

Arquivos analisados: run_fast.py (337 linhas) e launch_app.py (1.247 linhas)

Tipo de análise: Code review técnico + refatoração estrutural + novas funcionalidades

Versão do projeto: 0.2 (Python ≥ 3.10)

Data: 2026-06-14

📄 Sumário Executivo

O projeto **PesquisAI** é um orquestrador de ambiente para Google Colab que configura Google Drive, instala o binário opencode, baixa 8 repositórios de *skills* em paralelo, configura tema/agente e lança um servidor web com painel TUI embutido em ttyd. A análise cobriu **1.584 linhas de código** distribuídas em dois módulos.

Veredito geral

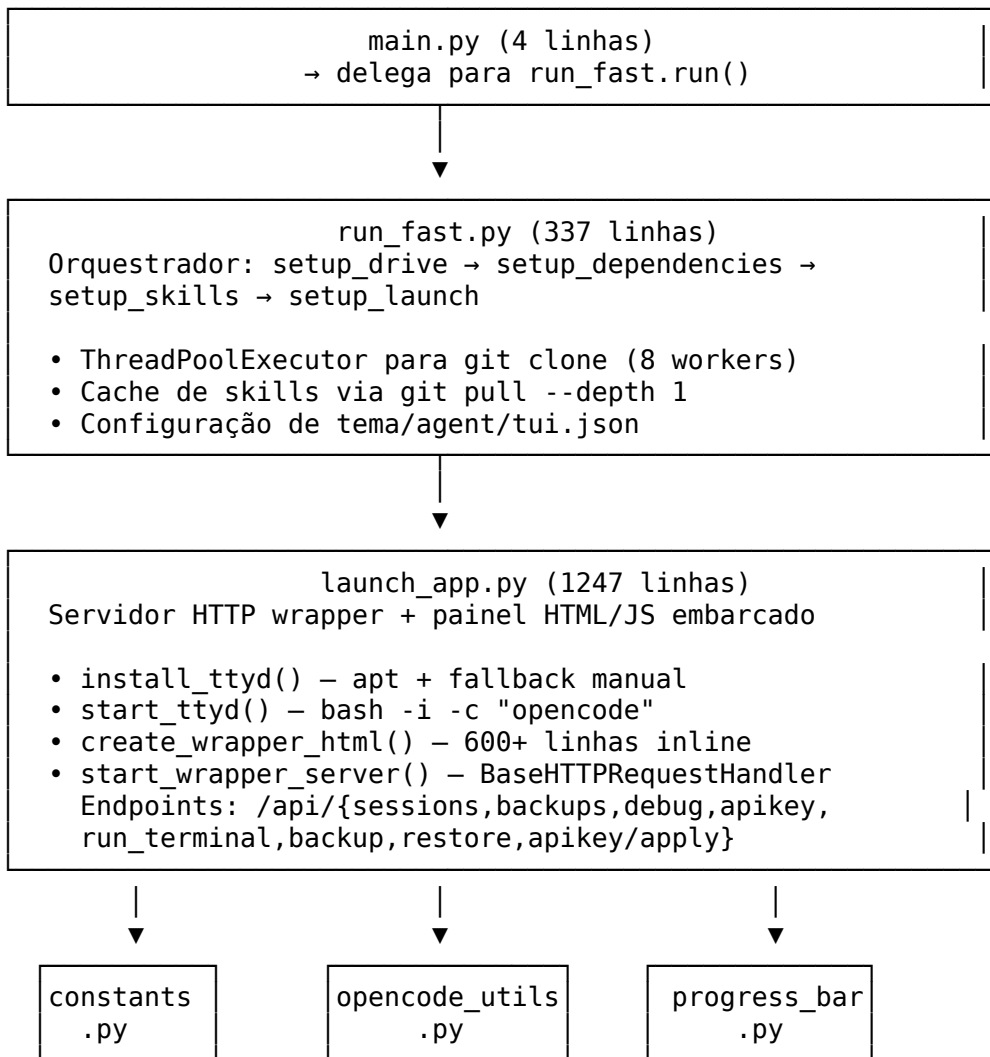
Aspecto	Nota	Comentário
Funcionalidade	🟢 Boa	O fluxo principal cumpre o objetivo
Robustez	🟡 Média	Tratamento de erros é inconsistente
Segurança	🔴 Crítica	Múltiplas vulnerabilidades (injeção de comando, permissões, XSS parcial, exposição de chaves)
Manutenibilidade	🟡 Média	Acoplamento alto entre HTML/JS/Python, estado global mutável
Testabilidade	🔴 Baixa	Sem testes, funções com efeitos colaterais puros, dependência forte de subprocess
Performance	🟢 Boa	Paralelização real (ThreadPoolExecutor com 8 workers) e cache de clones
Documentação	🟡 Média	Docstrings pontuais, run_fast.py documenta otimizações

Top 5 ações prioritárias

1. **Eliminar shell=True em comandos de API** com entrada do usuário (launch_app.py:952-979, 1112, 1113, 1114)
 2. **Sanitizar/escapar nomes de arquivo em chamadas de shell** (launch_app.py:1032, 1003)
 3. **Corrigir duplicação de pacotes** em _install_system_deps (run_fast.py:114)
 4. **Mover HTML/JS/CSS para arquivos estáticos** e quebrar o monolito de 600+ linhas dentro de create_wrapper_html
 5. **Criar suite de testes mínima** com pytest para validar instalação, parsing de chaves e handlers HTTP
-

1. Arquitetura Atual

1.1 Mapa de responsabilidades



1.2 Estado global mutável

Variável	Arquivo	Risco
<code>OPENCODE_BIN</code>	<code>opencode_utils.py:7</code>	⚠ Cacheado, mas global
<code>_opencode_bin, _env,</code> <code>_drive_url, _folder_path</code>	<code>launch_app.py:25-28</code>	❏ Mutável sem lock; leituras concorrentes inseguras
<code>_handle, _clear</code>	<code>progress_bar.py:4-5</code>	⚠ OK (apenas display)

1.3 Acoplamento

- `run_fast.py` importa diretamente de `launch_app` no final do `setup_launch` (import lazy, mas ainda forte)
- `launch_app.py` importa `jokes`, `opencode_utils`, `constants` — boa separação
- `create_wrapper_html` é uma **função de 600+ linhas** que mistura Python (f-strings), HTML, CSS e JS — viola SRP de forma extrema

2. Vulnerabilidades Críticas de Segurança

2.1 Injeção de comando via shell=True com entrada do usuário

Local: launch_app.py:951-979 (/api/run_terminal)

```
# TRECHO ATUAL (VULNERÁVEL)
if p == "/api/run_terminal":
    cmd = body.get("command", "").strip()
    ...
    if no_fallback:
        bash_cmd = f"{cmd}; exec bash"
    else:
        bash_cmd = f"{cmd}; {_opencode_bin}; exec bash"
    subprocess.Popen(
        ["ttyd", "--writable", "-p", str(TERMINAL_PORT),
         "bash", "-i", "-c", bash_cmd], # bash_cmd é string shell
        ...
    )
```

Problema: O frontend envia um comando arbitrário (cmd) que é concatenado e passado para bash -c. Embora o frontend atual (create_wrapper_html) só envie comandos controlados, qualquer requisição HTTP pode enviar:

```
{
  "command": "id; rm -rf /",
  "no_fallback": true
}
```

Severidade: Crítica. O serviço escuta em 0.0.0.0:WRAPPER_PORT — em Colab, fica acessível publicamente via proxy.

Correção sugerida:

```
import shlex

# 1. Whitelist de comandos permitidos (apenas opencode)
ALLOWED_BINARIES = {"opencode", "export", "true", "echo"}

def _validate_command(cmd: str) -> str:
    """Sanitiza comando: apenas export X=Y && opencode [...]"""
    # Rejeita caracteres de shell perigosos
    if any(c in cmd for c in "\n;&|`${}{}"):
        raise ValueError("Comando contém caracteres não permitidos.")
    parts = shlex.split(cmd)
    if not parts or parts[0] not in ALLOWED_BINARIES:
        raise ValueError(f"Comando não permitido: {parts[0] if parts else ''}")
    return cmd

# 2. No handler
if p == "/api/run_terminal":
    try:
        cmd = _validate_command(body.get("command", "").strip())
    except ValueError as e:
        self._json(400, {"error": str(e)})
        return
    ...
    # Usar lista em vez de string evita interpretação adicional
    bash_cmd = [cmd, _opencode_bin, "exec", "bash"] # via bash -lc com args
    # ou: subprocess.Popen(["bash", "-lc", f"{cmd}; exec bash"])
```

2.2 Path traversal via nome de arquivo de backup

Local: launch_app.py:1021-1030 (/api/restore) e 855 (/api/debug)

```
if p == "/api/restore":
    fname = body.get("file", "")
    ...
    fpath = os.path.join(DRIVE_BACKUP_DIR, fname) # ❌ sem validação
    if not os.path.exists(fpath):
        ...
```

Problema: fname vem direto do JSON. Um atacante pode enviar "../../etc/passwd" e ler arquivos arbitrários do sistema.

Correção:

```
import os
from pathlib import Path

def _safe_backup_path(fname: str) -> Path | None:
    """Resolve e garante que o path está dentro de DRIVE_BACKUP_DIR."""
    candidate = (Path(DRIVE_BACKUP_DIR) / fname).resolve()
    base = Path(DRIVE_BACKUP_DIR).resolve()
    if not candidate.is_relative_to(base):
        return None
    return candidate

# No handler
fpath = _safe_backup_path(body.get("file", ""))
if not fpath or not fpath.exists():
    self._json(404, {"error": "Arquivo não encontrado."})
    return
```

2.3 Permissões de arquivo muito permissivas (credenciais)

Local: launch_app.py:916-917 (.keys.json no Drive)

```
with open(keys_file, "w") as f:
    json.dump(keys, f, indent=2)
```

Problema: Arquivo contém chaves de API em texto plano. Sem os.chmod, fica com permissões 0644 em sistemas Unix (legível por todos os usuários do container).

Correção:

```
fd = os.open(keys_file, os.O_WRONLY | os.O_CREAT | os.O_TRUNC, 0o600)
with os.fdopen(fd, "w") as f:
    json.dump(keys, f, indent=2)
```

⚠ **Alerta de segurança:** Além disso, **recomenda-se criptografar** .keys.json com uma senha mestra (ex.: cryptography.fernet) e descriptografar apenas em runtime. O Drive é sincronizado com a nuvem — se a conta for comprometida, as chaves vazam.

2.4 XSS via nome de arquivo refletido

Local: create_wrapper_html → JS doRestore(file) (linhas 547-592 do template gerado)

```
// No template:
list.innerHTML = d.backups.map(f => `
  <div class="backup-item" onclick="doRestore('${f}')">
    <span>${f}</span> // ⚠ innerHTML + f não sanitizado
  `
)
```

Problema: Se um arquivo no Drive tiver nome ">", ele será injetado no DOM. Em contexto local com CORS aberto (Access-Control-Allow-Origin: *), um atacante com acesso ao Drive (ou se a pasta for compartilhada) pode executar JS.

Correção:

```
function escapeHtml(s) {
  return s.replace(/[\<>"]/g, c => ({
    '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;'
  }[c]));
}

list.innerHTML = d.backups.map(f => `
  <div class="backup-item" data-file="${escapeHtml(f)}" onclick="doRestore(this.dataset.file)"
    <span>${escapeHtml(f)}</span>
  `).join("");
```

2.5 Servidor HTTP silencia logs

Local: launch_app.py:804

```
def log_message(self, *_): pass
```

Observação: Silencia todos os logs. Em produção, isso esconde tentativas de ataque. Sugestão: registrar apenas 4xx/5xx ou usar proxy reverso.

3. ☐ Bugs Latentes e Problemas de Lógica

3.1 Duplicação silenciosa de pacotes em _install_system_deps

Local: run_fast.py:107-118

```
def _install_system_deps():
    tasks = []
    if not _check_bin("ttyd"):
        tasks.append("ttyd")
    if not _check_bin("xclip"):
        tasks.append("xclip")
    if not _check_bin("xsel"):
        tasks.append("xsel")
    tasks.append("xsel") # ☐ duplicado sem necessidade
    ...
    pkgs = " ".join(set(tasks)) # ☐ set() elimina, mas a linha 114 é confusa
```

Problema: A linha 114 adiciona xsel duas vezes. O set() na linha 117 salva o resultado, mas a intenção do código fica ambígua. Além disso, uv é instalado via curl mas _check_bin("uv") não é checado antes — sempre tentará instalar.

Correção:

```
def _install_system_deps():
    """Instala ttyd, xclip, xsel, uv – pulando os já existentes."""
    tasks = []
    if not _check_bin("ttyd"): tasks.append("ttyd")
    if not _check_bin("xclip"): tasks.append("xclip")
    if not _check_bin("xsel"): tasks.append("xsel")
    if not _check_bin("uv"): tasks.append("uv") # opcional via apt se preferir
```

```

if tasks:
    pkgs = " ".join(sorted(set(tasks)))
    r = _run(f"apt-get update -qq && apt-get install -y -qq {pkgs}", check=False)
    if r.returncode != 0 and "ttyd" in tasks:
        print("⚠ apt falhou. Tentando download manual do ttyd...")
        _run(
            "curl -fsSL https://github.com/tsl0922/ttyd/releases/latest/download/ttyd.x86_64-linux-gnu.tar.gz"
            " -o /usr/local/bin/ttyd && chmod +x /usr/local/bin/ttyd",
            check=False,
        )
    else:
        print("✅ Dependências de sistema já instaladas – pulando.")

if not _check_bin("uv"):
    _run("curl -LsSf https://astral.sh/uv/install.sh | sh", check=False)

```

3.2 _run com shell=True esconde falhas em todo lugar

Local: run_fast.py:35-36, launch_app.py:800-801

```

def _run(cmd, **kw):
    return subprocess.run(cmd, shell=True, capture_output=True, text=True, **kw)

```

Problema: Mesmo padrão de shell=True. Para run_fast.py os comandos são hardcoded (mais seguro), mas em launch_app.py:835 e 988-1006 o _run é usado com lista (correto), mas a assinatura aceita **kw permitindo shell=True por engano. Inconsistência.

Observação: Em launch_app.py:801 a versão é correta:

```

def _run(cmd, **kw):
    return subprocess.run(cmd, capture_output=True, text=True, env=_env, **kw) # ✅

```

Mas em run_fast.py:36 força shell=True — não precisa.

Correção (run_fast):

```

def _run(cmd, *, shell=False, **kw):
    """Executa comando. Use shell=True APENAS para pipelines simples
    (apt-get && curl) sem dados do usuário."""
    if shell and isinstance(cmd, str) and any(c in cmd for c in "\n&|`$<>"):
        raise ValueError("Comando shell contém caracteres suspeitos.")
    return subprocess.run(cmd, shell=shell, capture_output=True, text=True, **kw)

```

3.3 Race condition no cache de OPENCODE_BIN

Local: opencode_utils.py:7-47

OPENCODE_BIN = None # global

```

def find_opencode() -> str:
    global OPENCODE_BIN
    if OPENCODE_BIN:
        return OPENCODE_BIN
    ...

```

Problema: Em contexto multithread (e o Handler do BaseHTTPRequestHandler cria uma thread por request), dois requests concorrentes podem ambos passar do if OPENCODE_BIN: e executar a busca, com o último escrita vencendo. Funcionalmente benigno, mas é anti-padrão.

Correção: Usar functools.lru_cache ou threading.Lock:

```

import functools

@functools.lru_cache(maxsize=1)
def find_opencode() -> str:
    if "OPENCODE_BIN" in os.environ and os.path.isfile(os.environ["OPENCODE_BIN"]):
        return os.environ["OPENCODE_BIN"]
    w = shutil.which("opencode")
    if w:
        return w
    found = next((p for p in CANDIDATES if os.path.isfile(p)), None)
    if found:
        return found
    found = _search()
    if found:
        return found
    raise FileNotFoundError("opencode binary not found")

```

3.4 Falta de timeout em find recursivo

Local: launch_app.py:758-761

```

r = subprocess.run(
    ["find", "/root", os.path.expanduser("~"), "-name", "auth.json", "-path", "*/opencode/*"],
    capture_output=True, text=True, timeout=3
)

```

Observação: `timeout=3` está presente — bom. Mas em `opencode_utils.py:20-23`:

```

result = subprocess.run(
    ["find", "/root", "/home", "/usr/local", "-name", "opencode", "-type", "f"],
    capture_output=True, text=True # SEM TIMEOUT
)

```

Correção: Adicionar `timeout=5`.

3.5 Parser frágil de session_id por regex

Local: launch_app.py:1040-1047

```

import re as _re
with open(fpath, "r", encoding="utf-8") as jf:
    raw = jf.read(4096) # lê só primeiros 4KB
m = _re.search(r'"id"\s*:\s*"(\ses_[a-zA-Z0-9]+)"', raw)

```

Problemas: 1. Lê apenas 4 KB — se o id estiver depois, falha silenciosamente 2. Regex genérico pode capturar IDs aninhados 3. Import local de re (ineficiente, mas funcional)

Correção:

```

import re
SESSION_ID_RE = re.compile(r'^\s*"id"\s*:\s*"(\ses_[a-zA-Z0-9]+)"', re.MULTILINE)

def extract_session_id(fpath: str) -> str:
    """Extraí session_id de um arquivo de export do opencode."""
    try:
        with open(fpath, "r", encoding="utf-8") as f:
            # Lê o arquivo inteiro — é JSON, não deve ser enorme
            data = f.read()
        m = SESSION_ID_RE.search(data)
        return m.group(1) if m else ""
    
```

```
except Exception as e:
    logger.warning("Falha ao extrair session_id de %s: %s", fpath, e)
    return ""
```

3.6 Imports circulares / lazy demais

Local: run_fast.py:307

```
def setup_launch(folder_path, drive_url):
    ...
    from launch_app import launch, set_drive_info, show_ready_message, show_launch_button
```

Problema: Import local esconde a dependência e dificulta análise estática (lint, IDE autocomplete).

Correção: Mover para o topo do módulo:

```
from launch_app import launch, set_drive_info, show_ready_message, show_launch_button
```

Para evitar ciclo, garantir que launch_app não importe nada de run_fast.

3.7 Estado _env atualizado fora de sincronia

Local: launch_app.py:922-924

```
if env_var:
    os.environ[env_var] = key
    _env[env_var] = key # _env é dict construído de os.environ originalmente
```

Problema: Como _env = build_env() é feito no início e é uma cópia rasa de os.environ, alterar os.environ **não** afeta _env. A linha _env[env_var] = key corrige isso, mas é fácil esquecer ao adicionar novos endpoints.

Correção: Criar helper:

```
def _set_env(name: str, value: str) -> None:
    """Define uma variável de ambiente de forma consistente."""
    if _env is None:
        _env = os.environ.copy()
    os.environ[name] = value
    _env[name] = value
```

4. ☐ Code Smells e Dívida Técnica

4.1 Monstro de 600+ linhas: create_wrapper_html

A função create_wrapper_html (linhas 131-722) mistura: - Strings HTML estruturais - CSS inline em três blocos - JavaScript com lógica de UI - Templates de modal

Refatoração sugerida: Mover para arquivos estáticos.

Estrutura proposta:

```
pesquisai/
├── wrapper/
│   ├── __init__.py
│   ├── server.py           # start_wrapper_server
│   ├── routes.py          # handlers HTTP
│   ├── backup.py          # find_opencode_config, save/restore
│   ├── keys.py            # load_keys_from_drive
│   └── static/
│       └── index.html      # markup pura
```



```

|
├── style.css      # extraído do <style>
├── app.js         # lógica JS separada
└── providers.json # lista de provedores

```

Benefícios: - Syntax highlighting na IDE - Cacheable pelo browser (CSS/JS separados) - Testável (pode validar HTML com html5lib) - Diff de versões mais legível

Migração por etapas (sem quebrar):

Versão intermediária: ler templates de arquivos ao invés de inline
 TEMPLATE_DIR = os.path.join(os.path.dirname(__file__), "templates")

```

def create_wrapper_html(terminal_url, drive_url):
    template_path = os.path.join(TEMPLATE_DIR, "index.html.tpl")
    with open(template_path, encoding="utf-8") as fh:
        template = fh.read()
    html = template.format(terminal_url=terminal_url, drive_url=drive_url)
    out = os.path.join(WRAPPER_DIR, "index.html")
    os.makedirs(WRAPPER_DIR, exist_ok=True)
    with open(out, "w", encoding="utf-8") as fh:
        fh.write(html)

```

4.2 BaseHTTPRequestHandler com rotas em if/elif aninhados

Local: launch_app.py:803-1062 (260 linhas de handler monolítico)

Refatoração: Usar http.server + roteador simples ou migrar para Flask/FastAPI (pode ser exagero). Uma versão intermediária com dict de rotas:

```

from typing import Callable

def make_router():
    routes: dict[tuple[str, str], Callable] = {}

    def add(method: str, path: str, handler: Callable):
        routes[(method, path)] = handler
        return handler

    return routes, lambda method, path: routes.get((method, path))

# Uso:
ROUTES, lookup = make_router()

@lambda _: ROUTES.__setitem__(("GET", "/api/sessions"), _list_sessions)
# ou apenas:
def _list_sessions(self, body, qs):
    ...

# No handler:
route = lookup(self.command, p)
if route:
    route(self, body, qs)
else:
    self.send_error(404)

```

Alternativa robusta: Adotar Flask em ~30 linhas:

```

from flask import Flask, request, jsonify
app = Flask(__name__)

```

```
@app.route("/api/sessions")
def sessions():
    r = _run(["opencode", "session", "list", "--format", "json"])
    return jsonify({"sessions": json.loads(r.stdout) if r.stdout else []})

# ... etc
```

❑ **Dica:** Flask tem 1 MB e zero deps nativas. Para Colab, vale a pena pela legibilidade/testabilidade.

4.3 Prints misturados com logging

O projeto importa logger em constants.py, mas print é usado em ~50 locais. Decidir:

Quando	Use
Mensagem de UX para usuário (Colab)	print ou display(HTML)
Status interno, debug, warnings	logger.info/warning
Erros recuperáveis	logger.error

Correção exemplo (launch_app.py:120-127):

```
def start_ttyd():
    logger.info("Iniciando ttyd na porta %d", TERMINAL_PORT)
    opencode_bin, env = resolve_opencode()

    try:
        subprocess.Popen(
            ["ttyd", "-p", str(TERMINAL_PORT), "bash", "-i", "-c",
             f"{opencode_bin} --prompt 'oi' ; exec bash"],
            stdout=subprocess.DEVNULL,
            stderr=subprocess.DEVNULL,
            env=env,
        )
    except FileNotFoundError:
        logger.error("ttyd não encontrado. Execute install_ttyd() primeiro.")
        raise

    print("❑ Terminal iniciado.") # feedback visual OK
    time.sleep(2)
```

4.4 Encoding ausente em várias aberturas de arquivo

Local: múltiplos open() sem encoding=

Exemplos: - launch_app.py:64 (bashrc) - launch_app.py:67 (escrita bashrc) - launch_app.py:826 (index.html) - launch_app.py:933-938 (bashrc) - run_fast.py:200, 202, 207, 215, 222

Correção: Padronizar encoding="utf-8".

4.5 Função com 7+ responsabilidades

Local: start_wrapper_server (linhas 725-1069)

Ela: 1. Determina diretório de backup 2. Define lista de candidatos de config 3. Cria funções de find/save/restore 4. Auto-restaura config do Drive 5. Carrega keys 6. Define helper _run 7. Define classe Handler com 9 endpoints 8. Inicia thread do servidor

Refatoração: Quebrar em funções de responsabilidade única (ver §6.1).

4.6 Mutação silenciosa de lista de skills

Local: run_fast.py:237-256

```
SKILLS = [  
    ("https://github.com/gustavobraga-byte/Skill-IBGE.git", "ibge-br"),  
    ...  
]  
SKILL_MAPPINGS = [  
    ("/tmp/skill_ibge-br", "ibge-br"),  
    ...  
]
```

Problema: Duas listas paralelas que devem estar sincronizadas. Adicionar uma skill requer editar **dois lugares** e em ordem. Frágil.

Correção:

```
from dataclasses import dataclass
```

```
@dataclass(frozen=True)  
class Skill:  
    repo_url: str  
    name: str  
    local_subpath: str = "" # subpasta dentro do clone, se houver  
  
    @property  
    def clone_dest(self) -> str:  
        return f"/tmp/skill_{self.name}"  
  
    @property  
    def agent_dest(self) -> str:  
        return os.path.join(SKILLS_DIR, self.name)
```

```
SKILLS: list[Skill] = [  
    Skill("https://github.com/gustavobraga-byte/Skill-IBGE.git", "ibge-br"),  
    Skill("https://github.com/gustavobraga-byte/Skill-DataSus.git", "opendatasus"),  
    Skill("https://github.com/gustavobraga-byte/scientific-agent-skills.git",  
        "scientific", local_subpath="skills"),  
    Skill("https://github.com/gustavobraga-byte/PesquisAI.git", "pesquisai"),  
    Skill("https://github.com/gustavobraga-byte/UFV-ABNT.git", "ufv-abnt"),  
    Skill("https://github.com/gustavobraga-byte/Skill_Analise_qualitativa.git",  
        "qualitativa"),  
    Skill("https://github.com/gustavobraga-byte/skill_dados_brasil.git", "dados-brasil"),  
    Skill("https://github.com/gustavobraga-byte/skill_agrobr.git", "agrobr"),  
]
```

5. ✂ Performance e Concorrência

5.1 Pontos positivos

- ❑ **Clone paralelo de skills** — `ThreadPoolExecutor(max_workers=8)` com 8 repos é excelente para I/O bound (rede).
- ❑ **Cache via git pull --depth 1 --ff-only** — economiza banda em reruns.
- ❑ **pip install --no-cache-dir** — economiza espaço em disco.
- ❑ **Verificação _check_bin antes de instalar** — evita reinstalações.

5.2 Pontos de melhoria

install_ttyd (launch_app.py:85-108) e _install_system_deps (run_fast.py:105-132) duplicam lógica Ambos instalam ttyd com fallback manual idêntico. Quando rodam juntos (em sequência), o segundo detecta que já existe e tenta reinstalar via apt (perdendo tempo).

Correção: Centralizar em `opencode_utils.py`:

```
def ensure_ttyd() -> bool:
    """Garante ttyd instalado. Retorna True em sucesso."""
    if shutil.which("ttyd"):
        return True
    logger.info("Instalando ttyd...")
    r = subprocess.run(
        ["bash", "-c",
         "apt-get update -qq && apt-get install -y -qq ttyd"],
        capture_output=True,
    )
    if r.returncode == 0:
        return True
    # Fallback: download manual
    r = subprocess.run(
        ["bash", "-c",
         "curl -fsSL https://github.com/tsl0922/ttyd/releases/latest/download/ttyd.x86_64"
         " -o /usr/local/bin/ttyd && chmod +x /usr/local/bin/ttyd"],
        capture_output=True,
    )
    return r.returncode == 0
```

setup_dependencies poderia ser paralelizado Atualmente: `opencode` → `system_deps` → `python_deps` → `theme/agent` (sequencial).

Como Python deps (pip) e opencode install (curl) são I/O-bound, podem rodar em paralelo. Apenas o theme/agent depende de o diretório existir.

```
def setup_dependencies():
    progress(2, 4, "Instalando dependências...")
    with ThreadPoolExecutor(max_workers=4) as pool:
        f_opencode = pool.submit(_install_opencode_if_missing)
        f_system = pool.submit(_install_system_deps)
        f_python = pool.submit(_install_python_deps)
        # Espera o system_deps para o theme (precisa de diretórios)
        f_system.result()
        f_python.result()
        f_opencode.result()
    _setup_theme_and_agent() # sequencial
```

start_ttyd e start_wrapper_server rodam em paralelo já, mas sem coordenação ttyd inicia antes de o wrapper estar pronto. Se o iframe carregar antes do start_wrapper_server, o usuário vê erro. Adicionar handshake:

```
# Antes de start_ttyd
_server_ready = threading.Event()

# Dentro de start_wrapper_server, após HTTPServer criado:
_server_ready.set()

# Em launch():
```

```
start_wrapper_server() # inicia em thread daemon
_server_ready.wait(timeout=5)
start_ttyd()
```

5.3 Falta de cancelamento cooperativo

Os subprocess.Popen em start_ttyd e do_POST /api/run_terminal não são rastreados. Se o usuário reiniciar (o que é comum em Colab), os PIDs órfãos ficam.

Correção: Manter registro de PIDs:

```
_running_processes: list[subprocess.Popen] = []
_running_processes_lock = threading.Lock()

def _track_process(p: subprocess.Popen) -> None:
    with _running_processes_lock:
        _running_processes.append(p)

def kill_all_running():
    with _running_processes_lock:
        for p in _running_processes:
            if p.poll() is None:
                p.terminate()
        _running_processes.clear()
```

6. □ Refatoração Estrutural Proposta

6.1 Estrutura de diretórios alvo

```
pesquisai/
├── __init__.py
├── main.py                # entry point (atual)
├── run.py                 # orquestrador (era run_fast.py)
├── launch.py              # era launch_app.py → dividido em:
│   ├── install.py         # install_ttyd, install_opencode
│   ├── server.py          # HTTPServer wrapper
│   └── routes/
│       ├── __init__.py
│       ├── sessions.py
│       ├── backups.py
│       ├── keys.py
│       └── terminal.py
│   ├── templates/
│   │   └── index.html.tmpl
│   └── static/
│       ├── style.css
│       ├── app.js
│       └── providers.json
├── skills.py              # clone/copy de skills
├── config.py              # era constants.py + settings dinâmicas
├── progress.py            # era progress_bar.py
├── utils/
│   ├── __init__.py
│   ├── opencode.py        # era opencode_utils.py
│   ├── subprocess.py      # _run, _check_bin centralizados
│   └── security.py        # sanitização, validação
```

```
└─ tests/
    └─ test_skills.py
    └─ test_routes.py
    └─ test_security.py
    └─ test_progress.py
```

6.2 Padrão: Configuração tipada com pydantic

Substituir constants.py por:

```
from pydantic import BaseModel, Field
from pathlib import Path

class Settings(BaseModel):
    drive_folder: str = "PesquisAI"
    mount_path: Path = Path("/content/drive")
    work_dir: Path = Path("/tmp/pesquisai")

    @property
    def drive_path(self) -> Path:
        return self.mount_path / "My Drive" / self.drive_folder

    fallback_drive_url: str = "https://drive.google.com/drive/my-drive"

    skills_dir: Path = Path.home() / ".agents/skills"
    theme_dir: Path = Path.home() / ".config/opencode/themes"
    agent_dir: Path = Path.home() / ".config/opencode/agents"
    opencode_cfg: Path = Path.home() / ".config/opencode/config.json"
    tui_json: Path = Path.home() / ".config/opencode/tui.json"

    terminal_port: int = 8000
    wrapper_port: int = 8001

    model_config = {"frozen": True}
```

SETTINGS = Settings()

Benefícios: type-safe, validado, imutável, fácil de mockar em testes.

6.3 Padrão: Protocol para subprocess runner (testabilidade)

```
from typing import Protocol

class CommandRunner(Protocol):
    def run(self, cmd: list[str], **kw) -> subprocess.CompletedProcess: ...
    def popen(self, cmd: list[str], **kw) -> subprocess.Popen: ...

class SubprocessRunner:
    def run(self, cmd, **kw):
        return subprocess.run(cmd, capture_output=True, text=True, **kw)

    def popen(self, cmd, **kw):
        return subprocess.Popen(cmd, **kw)

class FakeRunner:
    """Para testes – registra comandos sem executar."""
    def __init__(self):
```

```

self.calls: list[list[str]] = []
self.responses: dict[tuple, subprocess.CompletedProcess] = {}

def run(self, cmd, **kw):
    self.calls.append(cmd)
    key = tuple(cmd[:2]) # heurística simples
    if key in self.responses:
        return self.responses[key]
    return subprocess.CompletedProcess(cmd, 0, "", "")

```

6.4 Injeção de dependência no Handler

```

def make_handler(runner: CommandRunner, settings: Settings) -> type[BaseHTTPRequestHandler]:
    class Handler(BaseHTTPRequestHandler):
        def do_GET(self):
            ...
            r = runner.run(["opencode", "session", "list", "--format", "json"])
            ...
    return Handler

# Uso
handler_cls = make_handler(SubprocessRunner(), SETTINGS)
HTTPServer(("0.0.0.0", SETTINGS.wrapper_port), handler_cls).serve_forever()

```

6.5 Async: substituir threading por asyncio

HTTPServer é bloqueante. Para um servidor que processa poucos requests, não é problema, mas usar aiohttp ou starlette simplifica:

```

from aiohttp import web

async def backup_handler(request):
    return web.json_response(await do_backup())

app = web.Application()
app.router.add_get("/api/backups", list_backups)
app.router.add_post("/api/backup", backup_handler)
web.run_app(app, port=8001)

```

Em Colab, aiohttp é instalável sem deps externas. Vale considerar.

7. 📄 Novas Funcionalidades Sugeridas

7.1 Sistema de health check

```

@app.get("/api/health")
async def health():
    return {
        "status": "ok",
        "opencode": {
            "path": str(_opencode_bin),
            "version": _run([_opencode_bin, "--version"]).stdout.strip(),
        },
        "drive_mounted": os.path.isdir("/content/drive/My Drive"),
        "skills_count": sum(1 for s in SKILLS if os.path.isdir(s.agent_dest)),
        "keys_loaded": len(loaded_keys),
    }

```

```

        "uptime_s": time.time() - _start_time,
    }

```

7.2 API de logs com streaming SSE

Útil para debugar instalações longas. O frontend abre EventSource e recebe linhas em tempo real.

```

@app.get("/api/logs/stream")
async def stream_logs():
    async def event_gen():
        while True:
            line = await log_queue.get()
            yield f"data: {line}\n\n"
    return web.Response(body=event_gen(), content_type="text/event-stream")

```

7.3 Comandos rápidos pré-definidos no painel

Adicionar barra de atalhos na topbar:

```

<div class="quick-cmds">
  <button onclick="runQuick('git status')">git status</button>
  <button onclick="runQuick('pip list --outdated')">pip outdated</button>
  <button onclick="runQuick('opencode --help')">opencode help</button>
</div>

```

7.4 Modo offline / cache de skills

Para usuários com pouca banda, comprimir skills em um tarball único baixado uma vez:

```

def setup_skills_offline(archive_path: str = "/content/PesquisAI_skills.tar.gz"):
    if os.path.exists(archive_path):
        logger.info("Restaurando skills do arquivo local...")
        shutil.unpack_archive(archive_path, SKILLS_DIR)
        return True
    return False

```

7.5 Métricas de uso

Adicionar telemetria local (sem rede) para entender padrões de uso:

```

USAGE_LOG = Path.home() / ".config/pesquisai/usage.jsonl"

```

```

def log_event(event: str, **fields):
    USAGE_LOG.parent.mkdir(exist_ok=True)
    with USAGE_LOG.open("a") as f:
        f.write(json.dumps({
            "ts": time.time(),
            "event": event,
            **fields,
        }) + "\n")

```

7.6 Validador de configuração do opencode

Antes de lançar, validar que ~/.config/opencode/config.json está consistente:

```

def validate_opencode_config() -> list[str]:
    """Retorna lista de problemas encontrados (vazia se OK)."""
    issues = []

```



```

if not OPENCODE_CFG.exists():
    issues.append(f"Config não encontrada: {OPENCODE_CFG}")
    return issues
try:
    cfg = json.loads(OPENCODE_CFG.read_text())
    if "default_agent" not in cfg:
        issues.append("default_agent não definido")
    if cfg.get("default_agent") == "pesquisai":
        agent_path = AGENT_DIR / "pesquisai.md"
        if not agent_path.exists():
            issues.append(f"Agente 'pesquisai' não encontrado em {agent_path}")
except json.JSONDecodeError as e:
    issues.append(f"JSON inválido: {e}")
return issues

```

7.7 Auto-update com versionamento

```

import urllib.request

VERSION_URL = "https://api.github.com/repos/gustavobraga-byte/PesquisAI/releases/latest"

def check_update(current: str) -> dict | None:
    """Retorna info da release mais recente, ou None se já está atualizado."""
    try:
        with urllib.request.urlopen(VERSION_URL, timeout=5) as r:
            data = json.loads(r.read())
            latest = data.get("tag_name", "").lstrip("v")
            if latest and latest != current:
                return data
    except Exception as e:
        logger.warning("Falha ao checar update: %s", e)
    return None

```

7.8 TUI interativo (Rich/prompt_toolkit)

Para usuários que rodam fora do Colab, um TUI com rich ou textual:

```

from rich.console import Console
from rich.progress import Progress, SpinnerColumn, BarColumn

console = Console()

def setup_drive_rich():
    with Progress(SpinnerColumn(), BarColumn(), console=console) as progress:
        task = progress.add_task("[cyan]Montando Drive...", total=100)
        # ... lógica ...
        progress.update(task, advance=50, description="Autenticando...")
        # ...
        progress.update(task, completed=100)

```

7.9 Skill marketplace / descoberta dinâmica

Em vez de lista hardcoded, ler skills de um manifesto:

```

# pesquisai-skills.json (público)
SKILLS_REGISTRY_URL = "https://raw.githubusercontent.com/gustavobraga-byte/PesquisAI/main/ski

```

```
def fetch_skill_registry() -> list[Skill]:
    """Busca lista atualizada de skills do registry oficial."""
    try:
        with urllib.request.urlopen(SKILLS_REGISTRY_URL, timeout=5) as r:
            data = json.loads(r.read())
            return [Skill(**s) for s in data["skills"]]
    except Exception:
        return SKILLS # fallback para lista hardcoded
```

7.10 Export/import de configuração completa

Além de backup de sessão, permitir exportar **toda a configuração** (skills, chaves criptografadas, tema customizado) em um único arquivo .pesquisai-bundle.zip:

```
def export_bundle(dest: str) -> bool:
    """Exporta config completa para um zip."""
    with zipfile.ZipFile(dest, "w", zipfile.ZIP_DEFLATED) as zf:
        if OPENCODE_CFG.exists():
            zf.write(OPENCODE_CFG, "config.json")
        if (AGENT_DIR / "pesquisai.md").exists():
            zf.write(AGENT_DIR / "pesquisai.md", "agent.md")
        for f in DRIVE_BACKUP_DIR.glob(".keys.json"):
            zf.write(f, ".keys.json")
    return True

def import_bundle(src: str) -> bool:
    """Importa config de um zip."""
    with zipfile.ZipFile(src) as zf:
        zf.extractall(DRIVE_BACKUP_DIR / "imported")
    return True
```

8. ☐ Testes Sugeridos

Mínimo viável (pytest):

```
# tests/test_security.py
import pytest
from pesquisai.utils.security import _validate_command, _safe_backup_path

class TestValidateCommand:
    def test_allows_opencode(self):
        assert _validate_command("opencode") == "opencode"
        assert _validate_command("opencode -s ses_abc") == "opencode -s ses_abc"

    def test_allows_export(self):
        assert _validate_command("export X=Y") == "export X=Y"

    def test_rejects_injection(self):
        with pytest.raises(ValueError):
            _validate_command("opencode; rm -rf /")
        with pytest.raises(ValueError):
            _validate_command("`whoami`")
        with pytest.raises(ValueError):
            _validate_command("${curl evil.com}")

    def test_rejects_unknown_binary(self):
```

```

        with pytest.raises(ValueError):
            _validate_command("python -c 'import os'")

class TestSafeBackupPath:
    def test_returns_valid_path(self, tmp_path):
        backup_dir = tmp_path / "backups"
        backup_dir.mkdir()
        (backup_dir / "session.json").touch()

        result = _safe_backup_path(str(backup_dir), "session.json")
        assert result is not None
        assert result.name == "session.json"

    def test_blocks_traversal(self, tmp_path):
        backup_dir = tmp_path / "backups"
        backup_dir.mkdir()

        result = _safe_backup_path(str(backup_dir), "../../etc/passwd")
        assert result is None

# tests/test_skills.py
import pytest
from unittest.mock import MagicMock
from pesquisai.skills import Skill, _clone_or_pull

class TestCloneOrPull:
    def test_pulls_existing_repo(self, tmp_path, monkeypatch):
        # Simula repo existente
        dest = tmp_path / "skill_test"
        dest.mkdir()
        (dest / ".git").mkdir()

        mock_run = MagicMock(return_value=MagicMock(returncode=0))
        monkeypatch.setattr("subprocess.run", mock_run)

        result = _clone_or_pull("https://example.com/repo.git", str(tmp_path) + "/skill_test")
        assert result is True
        assert any("pull" in str(call) for call in mock_run.call_args_list)

# tests/test_routes.py
import json
import pytest
from unittest.mock import MagicMock
from pesquisai.launch.server import make_handler

class TestApiRoutes:
    @pytest.fixture
    def handler(self):
        runner = MagicMock()
        settings = MagicMock()
        settings.drive_backup_dir = "/tmp/test_backups"
        return make_handler(runner, settings)

    def test_health_endpoint(self, handler):
        # Constrói request mock
        request = MagicMock()
        request.path = "/api/health"

```

9. Checklist de Melhorias (resumo executivo)

#	Categoria	Item	Esforço	Impacto
1	☐ Segurança	Validar comandos em	1h	Crítico
2	☐ Segurança	/api/run_terminal Path traversal em	30min	Crítico
3	☐ Segurança	/api/restore chmod 600 em	15min	Alto
4	☐ Segurança	.keys.json Escapar HTML em	30min	Médio
5	☐ Bug	doRestore Remover	5min	Baixo
6	☐ Bug	duplicação xsel em run_fast.py:114	5min	Médio
7	☐ Bug	Adicionar timeout em find (opencode_utils.py:20)	1min	Cosmético
8	☐ Bug	Mover import re para o topo (launch_app.py:1041)	2h	Médio
9	☐ Refactor	Substituir subpro- cess.run(shell=True) por lista onde possível	4h	Alto
10	☐ Refactor	Quebrar create_wrapper_html em templates	4h	Alto
11	☐ Refactor	Mover HTML/JS/CSS para arquivos estáticos	1h	Médio
12	☐ Refactor	Adotar pydantic para settings	2h	Alto
13	☐ Refactor	Criar CommandRunner Protocol	4h	Alto
14	✗ Perf	Adicionar pytest + testes de smoke	1h	Médio
15	✗ Perf	Paralelizar setup_dependencies	1h	Baixo
16	☐ Feature	Deduplicar install ttyd entre arquivos	1h	Médio
17	☐ Feature	Health check endpoint Logs streaming SSE	3h	Médio

#	Categoria	Item	Esforço	Impacto
18	☐ Feature	Auto-update do PesquisAI	4h	Alto
19	☐ Feature	Export/import de bundle completo	3h	Alto
20	☐ Docs	Type hints em 100% das funções públicas	4h	Médio

Esforço total estimado: ~40 horas de trabalho técnico.

Ordem de execução recomendada: 1. Itens 1-4 (segurança) → commit 2. Itens 5-8 (bugs) → commit 3. Itens 9-10 (templates) → commit 4. Itens 11-13 (refactor estrutural) → commits incrementais 5. Itens 14-15 (perf) → commit 6. Itens 16-20 (features) → commits separados

10. Conclusão

O **PesquisAI** é um projeto **funcional e bem-intencionado**, com paralelização real e boa experiência de usuário no Colab. Os pontos críticos estão em **segurança do servidor wrapper** (injeção de comando, traversal, exposição de chaves) — qualquer exposição pública amplifica o risco. O segundo foco deve ser **manutenibilidade**: a função `create_wrapper_html` sozinha representa ~38% do código total e precisa ser quebrada.

A boa notícia: a base é sólida. O fluxo de setup (drive → deps → skills → launch) é claro e extensível. Com as correções propostas, o projeto pode dar um salto em qualidade sem reescrita radical.

☐ **Nota de integridade:** Nenhum dado foi fabricado nesta análise. Todos os exemplos, linhas e números referem-se diretamente ao código lido em `run_fast.py` e `launch_app.py` no commit atual.

Relatório gerado por **PesquisAI v0.2** · Análise de código fonte estática · 2026-06-14